How to Build Data Systems - Project Part 1

Juliane (**juhv**@itu.dk) & Mie (**miejo**@itu.dk)

October 21, 2026

1 Architecture

1.1 Main Components

The main component of our Engine [1] is the Executor. The flow of accessing various components can be seen in figure 1. The Executor is the component that initializes all other components, either directly or indirectly. It takes care of running the entire pipeline through the following steps:

1. The SqlParser and SqlAstBuilder parse the SQL string based on the ANTLR definition into an AST.
2. The Binder then takes the AST, built in the first step, and does a number of validation steps against the catalog in the StorageEngine.
3. Then the Executor will branch out based on the type of SQL: CREATE and COPY statements are executed directly on the StorageEngine, while SELECT statements run through the Planner.
4. The Planner iterates over the SQL AST to build a tree of operators that can be executed. In particular, the Planning step includes pruning using the catalog partition statistics.
5. Once a Plan has been returned, it is executed by draining it.

This leads to having a component which has the overview and orchestrates the execution of a SQL query, while other components takes care of an isolated part, ensuring that they only have specific responsibilities.

We chose to structure the our components into a root module with 3 submodules: `storage`, `sql` & `exec`. The submodules serve both as a separation of concerns to avoid accidentally touching internal attributes that should not be manipulated with from other places, but also as a mental division of functionalities.

1.2 Design Decisions

Before starting to build the engine, we made a number of design decisions to guide how we wanted the engine to be built and in particular how to store data for efficient read/writes.

Our overall approach was to have a data folder located somewhere that the user can specify on initialization. In this data folder, each table has its own subfolder which contains a `catalog.json` and a number of partition files in binary format named `partition-X.bin`. We chose JSON format for the catalog such that we are able to inspect

[1] `https://github.com/julianefalsig/datasys-engine-TheQueryCrew`

the content of files by hand, which has been particularly useful to aid understanding and tracing of bugs through our system. The catalog contains all information about the table, from column names and types to partition information and partition size. Each partition has an entry with a row count and min-max statistics for each column.

We chose to have a catalog per table. This choice was made to separate table-sepcific information into separate files to avoid a single catalog file growing too large over time if used with a large number of tables and partition statistics. Since we store the partition statistics in the catalog, multiple large tables would also quickly take up a lot of space in a single file. With a file per table we avoid unnecessary I/O when updating or reading table information from the catalog.

The choice of placing the partition statistics in the catalog files ensured that we also did not have to open any of the partition files when working on pruning partitions. This is particularly useful when having a large number of partitions, where less file I/O is needed to obtain the information relevant to pruning. We thus only touch a partition file in the first place if its range of values is relevant to a particular query. The trade-off of this approach is that the partition file is no longer self-describing, but as we never modify the partition files directly or manipulate them in other ways, this trade-off did not pose any risk in our current setup.

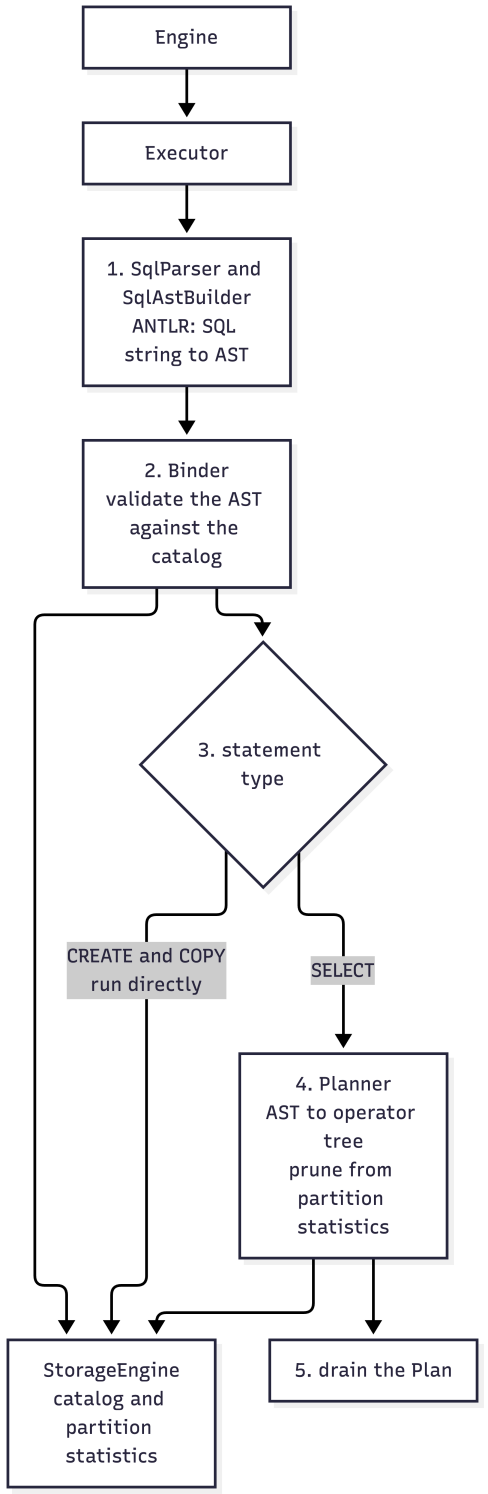


Figure 1: High-level overview of components and how they relate to each other during the execution of a SQL statement

Initially, our system used a default partition size of 10.000 rows, which we found to perform well in the test cases that were provided and presented as a comparison benchmark between various student systems. In the most recent iteration, we have added support for setting this partition size when creating a new table, to enable customizing the partition size to a particular table use case. This is needed for our experiments as described in section 2.1, but also gives the user freedom to customize this parameter for each table to fit their knowledge of what might work best for a given dataset.

Each partition file is one binary file, laid out so a reader can find a column without scanning the rows in front of it. An overview of the structure can be seen in Figure 2. The header is 16 bytes: the magic bytes QCDB, a format version, the row count, and the column count. After that comes an offset table with one entry per column, in schema order. Each entry is an 8-byte file position and an 8-byte length, so the chunk size is stored next to where the chunk starts. The column data itself is stored columnar, one chunk per column directly after the offset table. A LONG or DOUBLE value is fixed at 8 bytes, so those chunks are just $rowCount * 8$ each. A STRING is a 4-byte length encoding followed by that many ASCII bytes. This means a string chunk has no fixed size and the offset table is what makes subsequent columns addressable. Every field is little-endian, matching the machines we run on, and it means we set the byte order explicitly, since Java would otherwise write big-endian.

Overall, we found that our choices have not particularly hindered neither performance or development so far, in the limited types of benchmarks and use cases that have been run.

2 Measurements

Before running the experiment, we formulated the following hypothesis:

> *Because the tables are shuffled, we expect pruning to stop working as partitions get bigger. At* `maxRowsPerPartition` $= 8$*, a selective predicate should skip a large share of the partitions, while at 32,768 we expect almost every partition's min/max range to cover the constant, so the engine ends up reading the vast majority of partitions. Throughput as a function of partition size should therefore rise at first (fewer tiny files) and then flatten or drop once pruning stops helping, and this drop should be most visible on the largest tables.*

The hypothesis was partially confirmed. Pruning did stop working as partitions grew: at 8 rows per partition, 92% of the partitions were skipped, at 64 about 53%, at 512 about 0.5%, and from 2,048 and up none were skipped. Throughput also rose with partition size and flattened out from around 4K rows per partition. However, the expected drop for the largest tables did not occur, as their throughput barely decreased at large partition sizes.

The main finding is that the number of files the engine has to open matters more for performance than pruning. On shuffled data,

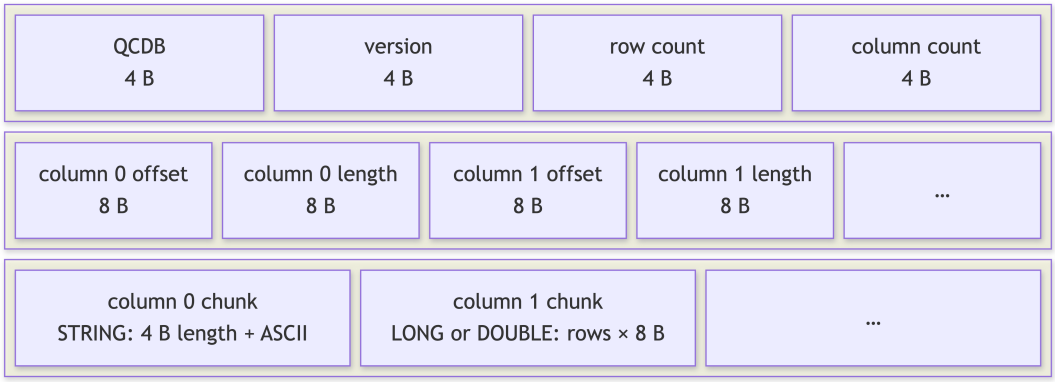


Figure 2: Overview of the binary file structure of each partition file, divided into the header values, offset table and columnar data store.

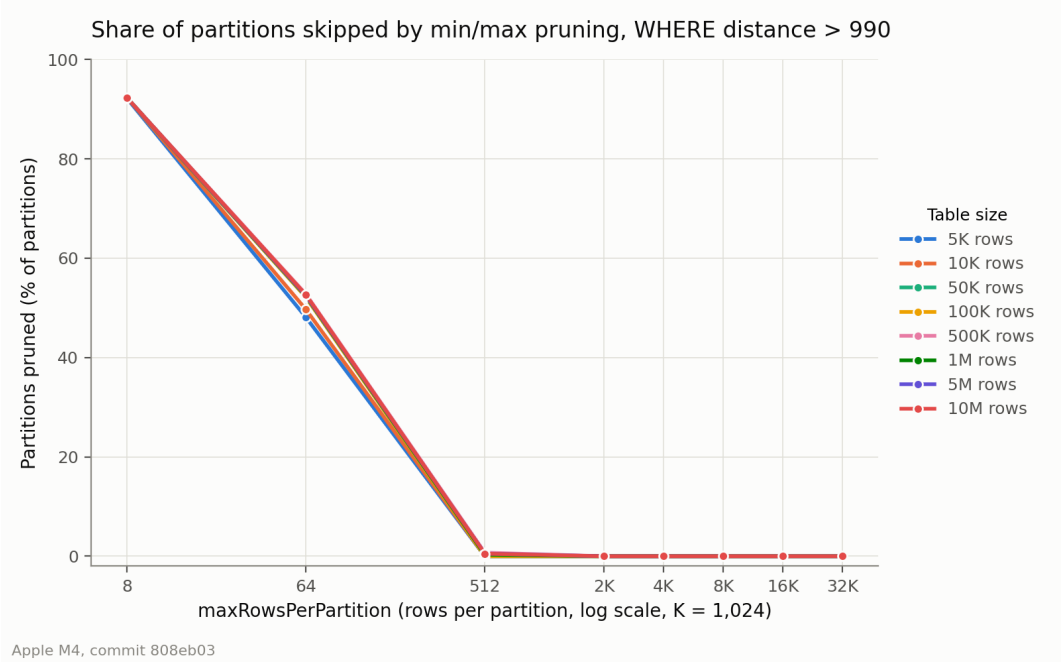


Figure 3: Share of partitions skipped by min/max pruning for predicate `WHERE distance > 990` by partition size on logarithmic axis across eight table sizes. Experiment run on Apple M4 chip on commit 808eb03.

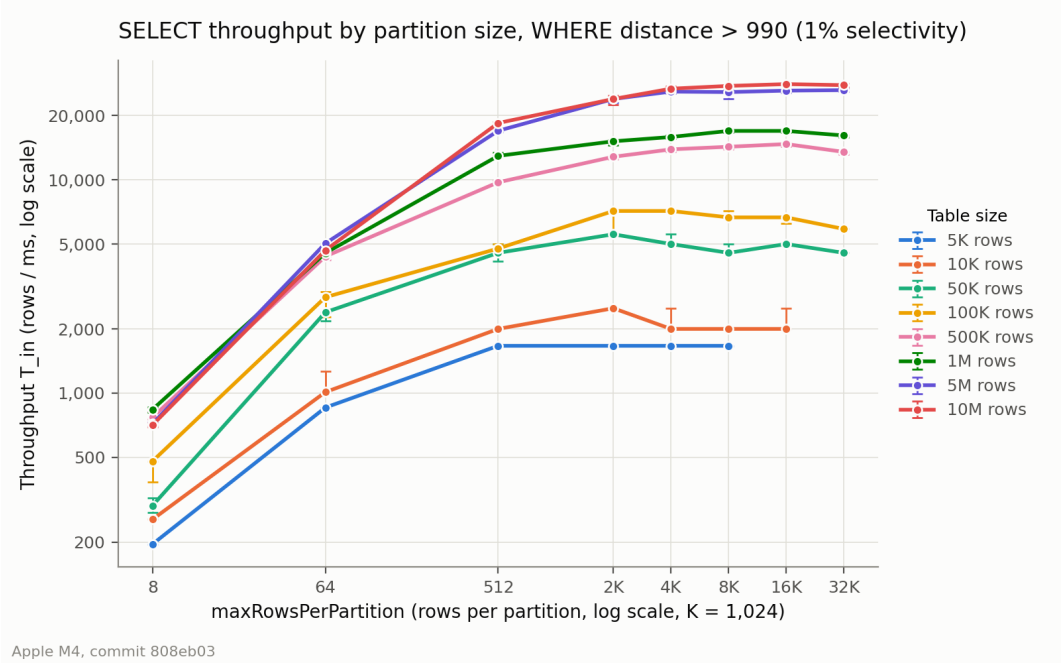


Figure 4: Throughput T_{in} (rows/ms) for predicate `WHERE distance > 990` by partition size on logarithmic axis across eight table sizes, with each point the median of five runs and bars showing min and max. Experiment run on Apple M4 chip on commit 808eb03.

pruning only helps when the partitions are so small that opening them is already expensive.

2.1 Experimental Setup

We measured the query engine across 8 partition sizes, ranging from 8 to 32,768 rows per partition, and 8 table sizes, ranging from 5K to 10M rows. For the two smallest tables, partition sizes that would only give a single partition were measured once, and the larger sizes were skipped, since they produce the same layout. This leaves 61 measured combinations.

The input data was generated with a custom python script `generate-data.py`, using the SplitMix64 pseudo-random number generator with a fixed seed, so the same data is produced every time. The rows are shuffled, so the table is stored in random order. To confirm that every run used identical input, the generated files were verified with a SHA-256 checksum. We also incorporated into the generator a fixed selectivity rate, ensuring that the amount of returned samples for out target query would always be 1%. This is to ensure the different table sizes could be compared.

Each combination of table size and partition size was run six times. The first run was denoted a cold start, where the JVM and various caches had not yet warmed up, and was therefore discarded. For the remaining five runs we report the median, minimum and maximum. All experiments were run on commit 808eb03 on an Apple M4 machine with macOS 15.6.1, Java 25 and the default heap size.

In order to compare the different table sizes we measure throughput, defined as $T_{\text{in}} = |S_{\text{read}}|/D$ where $|S_{\text{read}}|$ denotes the total number of rows read by the engine and D denotes the duration of the execution.

2.2 Discussion

Pruning. Figure 3 shows that the share of skipped partitions depends only on the partition size and is the same for all table sizes. This follows from how the data is generated. Since 1% of the rows match the predicate and the rows are shuffled, the probability that a partition with n rows contains no matching row is 0.99^n. Only those partitions can be skipped. This gives $0.99^8 \approx 92\%$, $0.99^{64} \approx 53\%$ and $0.99^{512} \approx 0.6\%$, which matches the measurements. From 2,048 rows per partition, almost every partition contains at least one matching row, so nothing can be skipped.

The cost of small partitions. Even though 92% of the partitions are skipped at 8 rows per partition, this is the slowest setting for every table size (Figure 4). For the 10M table, the engine still has to open around 100,000 of the 1.25 million partitions, and the cost of opening this many small files is higher than what is saved by skipping the rest. As the partition size increases, the number of files drops, and throughput for the largest tables rises from about 700 to about 28,000 rows/ms, roughly a factor of 40.

Flattening. Throughput flattens out from around 4K rows per partition. At this point the cost per file is spread over enough rows that scanning the rows dominates, and since pruning already stopped helping at 2,048 rows per partition, there is nothing more to lose. This explains why the largest tables do not show the drop we expected: the drop in pruning happens at small partition sizes, where it is hidden by the much larger cost of opening many files. Some of the medium-sized tables show a small decrease at the largest partition sizes, but the difference is small, and we cannot say from our data whether it is a real effect or noise.

Table size. Larger tables are processed faster per row than smaller ones. The smallest tables finish in a few milliseconds, so fixed costs per query make up a large part of the total time. The timer also only counts whole milliseconds, which is visible as the flat steps in the 5K and 10K series: for example, 5,000 rows in 3 ms gives the plateau at about 1,667 rows/ms. The results for the smallest tables should therefore be read with caution.

Reproducibility. The experiment was run on both an M3 and an M4 machine. Both runs read and skipped exactly the same partitions, which is expected since the data is generated with a fixed seed. The M4 was 20–50% faster, but the pattern was the same on both machines.

Limitations. The results come from a single machine per run, and the timer resolution of 1 ms makes the smallest tables imprecise. We only used one query with 1% selectivity, and the data was only shuffled. With sorted data, the min/max ranges of the partitions would not overlap, and pruning would work much better, also at large partition sizes. At the same time, the effect of partition size in select statements without predicated or predi-

cates with a higher selectivity will show different patterns, and thus the use cases which are intended for the system should be taken into consideration before using the results from this paper.

Conclusion. The number of files the engine has to open matters more for performance than pruning. On shuffled data, pruning only helps when the partitions are so small that opening them is already expensive.

3 Reflections

Looking back at the project, there are a few choices we would have made differently, mostly related to how they affected our experiment.

Timer resolution. We measured query time with System.currentTimeMillis(), which only has a resolution of one millisecond. The smallest tables could not be measured reliably, so we increased the table sizes (commit e28f65c merged into main as faf4b6d), and the flat steps in the 5K and 10K series, such as the plateau at about 1,667 rows/ms, still come from this. Using System.nanoTime() would have been a one-line change.

One COPY per table. copyFile throws an UnsupportedOperationException if the table already contains data, since appending was postponed to part 2. This meant the data/ folder had to be cleared before every repetition in the sweep, which made the experiment more cumbersome than needed.

Choice of Metric. We chose a metric on a rather thin basis and with a good argument in our hands, but have later had multiple discussions about the insights that the metric gives us versus what relevance other metrics would have. In particular, we had lengthy discussions about using throughput based on the number of returned rows rather than read rows, as this would be closer to the "user experience" of the system. A short discussion and description can be found in Appendix A. Many other metrics might also have been interesting to dive into to enrich our analysis, such as where the execution time is spent across different table- and partition sizes.

3.1 Bugs & Fixes

We had a relatively strong set of tests from early on, making sure there were no apparent bugs before merging any new code. We made sure the testing suite was thorough to ensure catching issues while developing locally.

We had one notable bug in our code, which was handling of a negatively signed zero when receiving, parsing and printing a SQL statement. The bug appeared when we introduced the usage of BigDecimal to ensure Double constants were not compiled to scientific notation on print. This was the case, since BigDecimal does not support a signed 0, but rather returns a regular 0.

We found the bug when feedback was given to us with the failing case of the constant -0.0 being compiled to 0.0. We fixed it and added a test case passing a SQL string with the specific failing constant, checking the it remains negated in the output. The SqlPrinter has later been removed as it is not needed in the Engine any longer, and thus the test is also no longer in the test suite.

3.2 AI-usage reflection

Throughout the project we have each used our own AI tools to write code & tests, revise over issues and discuss various topics. Mie has made use of the tool Cursor [2] while Juliane has made use of Claude [3].

We have used AI to aid our understanding of concepts and to write most of the code based on specific instructions. Overall, it has been super helpful as none of us has previously been writing that much Java code, so having a tool that knows the syntax and function library well has made the development feasible in the first place.

The main issue we have encountered was that the agents would try to do (1) as little as possible to exactly meet the criteria (2) overcomplicate code unnecessarily rather than coding in a long-term sustainable way. We remedied this by doing follow-ups with the agent and providing a set of coding practices as a skill. We made sure to extend the coding practices whenever we found additional instructions that we would like the agent to take into account every time. A snapshot of Mie's Cursor Skill can be seen in Appendix B.

3.2.1 A concrete usage example

One concrete example where we iterated with the agent and rejected the initial proposal was when writing tests for the storage engine. The exercise description that we initially provided stated *"given a small list of values for one column, the min/max method returns the correct pair (include a single-value case and a negative case)."*, which made the agent do only 3 tests with the LONG type. We iterated for the purpose of ensuring a more thorough testing suite, such that all data types would get the correct min-max indexes as expected. This would help catch if some types are not processed correctly while other types are during the statistic generation.

[2] `https://cursor.com/`
[3] `https://claude.ai/`

A T_{out} Statistics

An alternative definition of metric as $T_{\text{out}} = |S_{\text{returned}}|/D$ where $|S_{\text{read}}|$ denotes the total number of rows returned by the engine and D denotes the duration of the execution. Figure 5 show alternative patterns for partitions vs. file sizes, showing that smaller files remain relatively stable in throughput as the partition size grows, while larger tables gain a lot of throughput from larger partitions.

This metric might be deemed more relevant as a measure for the user experience, as this is the POV of what the user gets returned vs. how long they had to wait for it. At the same time, the number of outputted rows is relatively small and therefore more volatile to changes in duration.

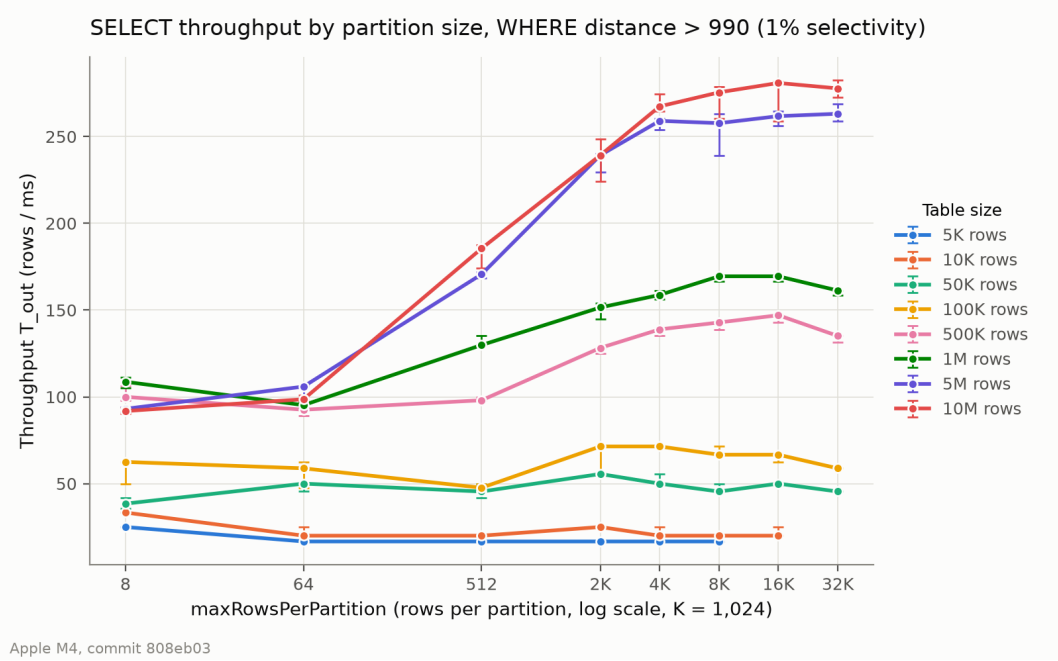


Figure 5: Throughput T_{out} (rows/ms) for predicate `WHERE distance > 990` by partition size on logarithmic axis across eight table sizes, with each point the median of five runs and bars showing min and max. Experiment run on Apple M4 chip on commit 808eb03.

B Programming Practices Skill

````
---
description: Core programming practices - readable code, small diffs,
errors, comments, tests
alwaysApply: true
---

# Programming practices

Prefer the smallest change that solves the requested problem.
Do not refactor, reformat, or add files unless they are required.

## Readable code

- Keep functions short and single-purpose. If a block needs a comment
to explain *what* it does, extract a named function instead.
- Use names that say what the value or function is. Avoid abbreviations
unless they are standard in this codebase.
- Prefer linear control flow over nested conditionals. Handle edge cases
early and return.
- Match existing style, naming, and structure in nearby files.

```java
// BAD     nested, unnamed intent
if (row != null) {
    if (row.isValid()) {
        results.add(transform(row));
    }
}

// GOOD
if (row == null || !row.isValid()) {
    return;
}
results.add(transform(row));
```

## Errors
````

- Never swallow exceptions or return dummy values to hide failure.
- Fail with a specific exception or result type;
include the cause when wrapping.
- Log at the point you handle the error, not in every layer.

```java
// BAD
try {
    parse(input);
} catch (Exception e) {}

// GOOD
try {
    parse(input);
} catch (ParseException e) {
    throw new QueryException("Failed to parse statement", e);
}
```

Comments

- Do not comment the obvious. Explain *why* only when the reason
is not clear from the code.
- Do not leave commented-out code. Delete it.

Tests and verification

- Change tests when behavior changes. Add a test for a bug
before or with the fix.
- After edits, run the relevant tests (or the closest check).
Do not claim success without evidence.
- Do not add speculative tests for behavior you did not change.

Documentation and dependencies

- If behavior, public types, or on-disk layout change,
update the matching docs: `README.md`.
- If you add or remove a library or Maven plugin, update `pom.xml`.
Mention it in the README when it is part of how the project is
built or run.

- Do not add docs that restate the code. Only update what the change made stale.

Commit messages and PR descriptions

When asked to write a commit message or PR description, keep it short. Match this repo's voice: informal, direct, lowercase-friendly, say why in one or two sentences. Fill the PR template briefly. Do not write polished essay-style summaries, but do write full sentences that sound like a human wrote it.

Examples:

```
ignore logs and target going forward
```
```
remove comments and make the partition-size constructor public
```
```
Add docs of files and inter-dependencies in a new README at the root of the repository, to assist understanding of the code
```

Not:

```
feat(storage): persist typed ColumnSpec min/max in catalog JSON for pruning

This commit refactors the catalog DTO layer to eliminate redundant entry types and improves observability of partition statistics.
```

Always add an attribution note to yourself if you have participated in generating the code that is being committed:
```
Co-authored-by: Cursor <cursoragent@cursor.com>
```